L-системы

L-системы

Идея: рецепт для роста

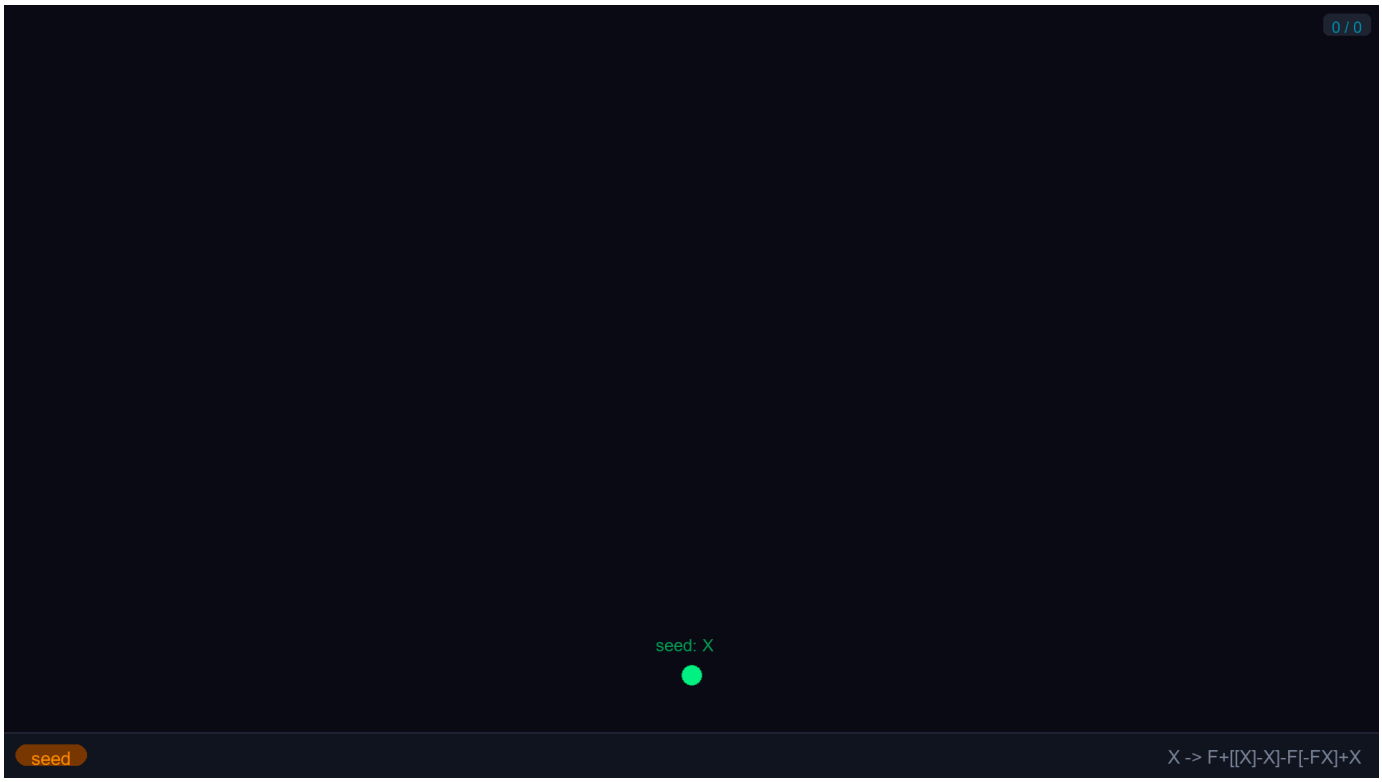
Все методы процедурной генерации, которые мы рассматривали до сих пор, работают с **сеткой**. Случайное блуждание прорезает тоннели, BSP делит пространство на комнаты, клеточные автоматы сглаживают шум. Но как сгенерировать **дерево**? Куст? Коралл? Молнию? Это не сетка - это ветвящаяся структура.

Ну, можно нарисовать дерево вручную - но... Допустим, нам нужен лес из 50 деревьев на фоне платформера - рисовать каждое руками? Можно рандомить углы и длины веток - но без правил получится каша, а не лес.

А что, если мы опишем форму не как набор тайлов, а как **рецепт роста**? Начинаем с зерна, применяем правила замены - и на каждом шаге структура становится сложнее. Как настоящее растение: из ростка - ствол, из ствола - ветки, из веток - побеги.

Именно так работают **L-системы** (Lindenmayer systems) - формальные грамматики, которые описывают рост через итеративную замену символов.

L-системы придумал биолог Аристид Линденмайер в 1968 году для моделирования роста водорослей. Потом оказалось, что тот же подход прекрасно генерирует деревья, кусты, снежинки, города и даже музыку.



Формальная грамматика

Ок. L-система - это, грубо говоря, три вещи:

1. **Аксиома** - начальная строка (зерно). Например: `F`
2. **Правила замены** - как менять каждый символ на новую строку. Например: `F → F+F-F-F+F`
3. **Количество итераций** - сколько раз применяем правила.

Каждая итерация проходит по строке и заменяет **все** символы одновременно (параллельная замена - это ключевое отличие от обычных грамматик Хомского).

Пример - квадратичная кривая Коха (угол поворота 90°):

- Аксиома: `F`
- Правило: `F → F+F-F-F+F`

Раскручиваем:

```
Итерация 0: F
Итерация 1: F+F-F-F+F
Итерация 2: F+F-F-F+F+F+F-F-F+F-F+F-F-F+F-F+F-F-F+F+F+F-F-F+F
...
```

Строка растёт экспоненциально. Итерация 6 для этого правила - уже тысячи символов, а итерация 9-10 - миллионы (и ваша программа это почувствует). На практике 4-6 итераций - разумный предел. Но нам и не нужно читать эту строку - нам нужно её **нарисовать**.

Turtle-графика: строка → рисунок

Каждый символ строки - это команда для "черепашки" (turtle), которая ходит по экрану и рисует линию за собой:

Символ	Действие
F	Шаг вперёд с рисованием линии
G	Тоже шаг вперёд с рисованием (альтернативный символ)
+	Поворот влево на угол δ
-	Поворот вправо на угол δ

Зачем два символа для одного действия? Некоторые L-системы (как треугольник Серпинского) используют два рисующих символа с **разными правилами замены** - F разворачивается по одному правилу, G по другому, но оба рисуют линию.

Угол δ - параметр. Для кривой Коха δ = 90°. Для снежинки Коха δ = 60°. Меняем угол - меняем форму.

Т.е. мы превращаем строку символов в набор команд: "иди, поверни, иди, поверни...". Черепашка рисует - и получается фрактал.

Python C#

```python
import math


def expand(axiom, rules, iterations):
    """Apply L-system rules to axiom n times."""
    s = axiom
    for _ in range(iterations):
        s = ''.join(rules.get(ch, ch) for ch in s)
    return s


def turtle_draw(lstring, angle_deg, step, start_x, start_y):
    """Interpret L-system string as turtle commands. Returns line segments
    x, y = start_x, start_y
    a = -90  # start pointing up
    segments = []

    for ch in lstring:
        if ch in ('F', 'G'):  # both symbols draw forward
            nx = x + step * math.cos(math.radians(a))
```

```
            ny = y + step * math.sin(math.radians(a))
            segments.append((x, y, nx, ny))
            x, y = nx, ny
        elif ch == '+':
            a -= angle_deg  # turn left
        elif ch == '-':
            a += angle_deg  # turn right

    return segments
```

Ветвление: деревья и кусты

Кривая Коха красивая, но это линия - без ветвлений. Настоящие растения **ветвятся**. Для этого добавляем два символа:

Символ	Действие
`[`	Сохранить текущую позицию и угол (push в стек)
`]`	Вернуться к сохранённой позиции (pop из стека)

Это превращает черепашку в "исследователя": она идёт вперёд, на развилке запоминает, где стоит (`[`), рисует ветку, возвращается обратно (`]`), и идёт рисовать следующую ветку.

Классическое правило для растения:

- Аксиома: `X`
- Правила: `X → F+[[X]-X]-F[-FX]+X` , `F → FF`
- Угол: 25°

`X` - ростовой символ. Он не рисует линию, а только размножается. `F` - рисующий символ, который тоже удваивается с каждой итерацией (ствол становится длиннее).

Модификация `turtle_draw` : добавьте стек. При встрече `[` - сохраняете текущие `(x, y, angle)` через `push` . При `]` - восстанавливаете через `pop` . Символ `X` - ростовой, не рисует линию, просто пропускается. Всё остальное (`F` , `G` , `+` , `-`) работает как раньше.

5 итераций - и из одного символа `X` вырастает полноценное дерево. Стек - ключевая структура данных: без него ветвление невозможно.

Один код - разные правила

Вот где L-системы показывают свою мощь. **Тот же самый код** (expand + turtle) с разными правилами и углом генерирует совершенно разные формы:

Название	Аксиома	Правила	Угол	Результат
Кривая Коха	`F`	`F → F+F-F-F+F`	90°	Зубчатая линия
Треугольник Серпинского	`F-G-G`	`F → F-G+F+G-F`, `G → GG`	120°	Фрактальный треугольник
Куст	`X`	`X → F-[[X]+X]+F[+FX]-X`	22.5°	Раскидистый куст
Сорняк	`X`	`X → F[+X]F[-X]+X`	20°	Тонкий сорняк

Это ключевой паттерн: **один алгоритм, разные параметры → разные результаты**. Тот же подход работает в процедурной генерации (разные seed'ы), в ИИ врагов (разные параметры FSM), в частицах (разные настройки эмиттера).

Стохастические L-системы

Хм, все примеры выше - детерминированные: одни и те же правила всегда дают одну и ту же форму. Но в природе два дерева одного вида выглядят похоже, но не одинаково.

Решение: **стохастические правила**. Вместо одного правила для символа - несколько вариантов с вероятностями:

```
X → F+[[X]-X]-F[-FX]+X    (60%)
X → F-[[X]+X]+F[+FX]-X    (40%)
```

Каждый раз, когда мы заменяем `X`, бросаем кость и выбираем правило. Результат: каждый запуск даёт немного другое дерево, но в рамках одного "вида".

Модификация `expand`: вместо одного правила для символа - список пар `(вероятность, замена)`. При замене символа бросайте случайное число от 0 до 1, накапливайте вероятности и выбирайте первый вариант, где кумулятивная сумма превысила число. Формат правил: `'X': [(0.6, 'F+[[X]-X]-F[-FX]+X'), (0.4, 'F-[[X]+X]+F[+FX]-X')]`.

Не забудьте про случайности: seed фиксирует конкретное дерево, а стохастические правила - это по сути взвешенный выбор из вариантов.

Стохастические L-системы удобны для генерации "леса": один набор правил, разные seed'ы - и каждое дерево уникально, но все они одного "вида".

Плюсы и минусы

Плюсы:

- **Компактность.** Несколько строк правил описывают структуру из тысяч сегментов. Одна L-система = целый лес.
- **Бесконечная детализация.** Хочешь больше деталей - добавь итерацию. Масштабируется без дополнительного дизайна.
- **Вариативность.** Стохастические правила + разные seed'ы = бесконечное разнообразие.
- **Простота кода.** Expand + turtle - 30-40 строк на любом языке.

Минусы:

- **Нет контроля формы.** Нельзя сказать "дерево высотой 200 пикселей с 5 ветками". Результат определяется правилами косвенно.
- **Чисто визуальная генерация.** L-система не знает о коллизиях, не учитывает окружение. Дерево может вырасти сквозь стену.
- **Не для уровней.** В теории, можно. Но есть альтернативы получше - BSP, WFC, клеточные автоматы и т.д.

Формальные грамматики и фракталы

L-системы - это **формальные грамматики**. Немного об их связи с теорией формальных языков или компиляторов:

- **Параллельная подстановка** - ключевое отличие от грамматик Хомского (где заменяется один символ за шаг). L-системы заменяют все символы одновременно - как клетки, делящиеся в организме.
- **Контекстно-свободные** - базовые L-системы: правило для F не зависит от соседних символов. Но существуют контекстно-зависимые варианты (1L-, 2L-системы), где правило для F зависит от того, что стоит слева и справа.
- **Черепашья интерпретация** - по сути, это маленький интерпретатор. Строка = программа, символы = инструкции, стек = вызовы функций. Тот же принцип, что и в языках со стековой ВМ.
- **Фракталы** - многие L-системы порождают самоподобные структуры. Кривая Коха, треугольник Серпинского, кривая дракона - всё это L-системы.

SpeedTree - middleware для генерации деревьев в AAA-играх - использует идеи L-систем внутри. The Witcher 3, Skyrim, Far Cry - все используют подобные алгоритмы для процедурной растительности.

Что дальше?

Что	Где подробнее
Другие методы генерации уровней	Процедурная генерация

Что	Где подробнее
Тайлы с правилами соседства	Wave Function Collapse
Seed и стохастика	Случайность и вероятность
Переходы с вероятностями вместо правил	Цепи Маркова
Эволюция параметров L-системы	Генетические алгоритмы
Рисование сегментов на экране	Камера и отсечение

Powered by Wiki.js